

Week 6: Flutter Routes

Dr M AOUDE

Lecture notes

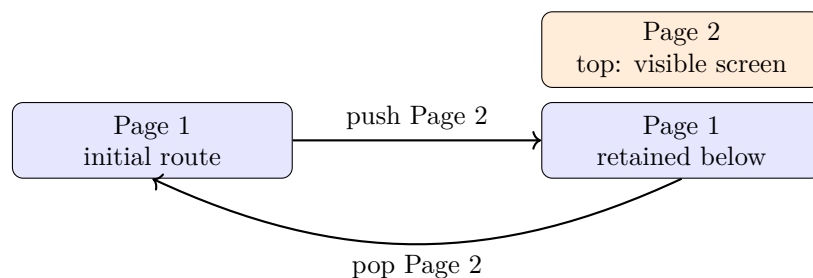
Objectives. By the end of this lecture you will be able to build an application with several screens; explain the route stack; open and close a screen with `Navigator.push` and `Navigator.pop`; choose the correct `BuildContext`; use icons in navigation buttons; pass an object through `RouteSettings`; retrieve it with `ModalRoute`; and manage text controllers in a small employee salary application.

This lecture follows the Flutter Routes slides supplied for the course, renumbered as **Week 6**. It builds on week 5 (state, text input, controllers and validation). The examples below are self-contained: create a Flutter project, then place the listed files in its `lib` directory. The first application and the second application are separate versions of the same project; replace their files when moving to the second example.

1 Screens, routes and the Navigator

A screen usually has its own `Scaffold`, an `AppBar`, and a body. A **route** describes an entry managed by a `Navigator`; in our examples that entry displays a screen widget. `MaterialPageRoute` wraps the widget and provides a Material page transition.

`MaterialApp(home: ...)` installs the first screen and a navigator. The navigator keeps a **stack** of routes: the most recently pushed route is the one on top. This is last in, first out (LIFO).



Operation	Effect
<code>push(route)</code>	Adds a route above the current route and displays it.
<code>pop()</code>	Removes the top route and reveals the route beneath it.
<code>MaterialPageRoute</code>	Builds the destination widget and manages its transition.

For these routes, the previous screen normally remains mounted while covered; its text fields and state are retained. It is not necessarily visible, and keeping it mounted does not mean all animation or background work is paused. Returning with `pop` reveals that screen; pushing another Page 1 would create a new route instead.

1.1 Passing data with routes

The route stack explains how we move between screens, but a useful app also sends data when it moves. We often want to say: “show the details for this item” or “show the order for this selected category”. The sender can pass arguments in `RouteSettings.arguments` using a `Map` or by passing an object from the same model layer. The receiving screen reads them with `ModalRoute.of(context)?.settings.arguments`; after a type check, it can use that data to build the next page. In other words, the route tells Flutter where to go, while the arguments tell it what data to show there. A simple pattern is:

```
1 Navigator.of(context).pushNamed(
2   '/selected',
3   arguments: {'id': item.id, 'title': item.name},
4 );
5
6 final arguments = ModalRoute.of(context)?.settings.arguments as Map<String,
7   String>;
8 final id = arguments?['id'];
```

This is the same idea used in a shopping app, a list/details app, or a restaurant menu where the chosen items are passed to a separate page.

2 First application: two screens

2.1 The entry point: main.dart

Create a project with `flutter create week06_routes`, open it in your editor and replace `lib/main.dart`. Add `page1.dart` and `page2.dart` beside it. Run with `flutter run` after choosing a device.

```
1 import 'package:flutter/material.dart';
2 import 'page1.dart';
3
4 void main() => runApp(const MyApp());
5
6 class MyApp extends StatelessWidget {
7   const MyApp({super.key});
8
9   @override
10  Widget build(BuildContext context) {
11    return const MaterialApp(
12      debugShowCheckedModeBanner: false,
13      title: 'Week 6: Flutter Routes',
14      home: Page1(),
15    );
16  }
17 }
```

2.2 Page 1: push a route

Both screens in this first example are stateless: neither owns changing data. A navigation callback does not by itself require a `StatefulWidget`.

```
1 // page1.dart
2 import 'package:flutter/material.dart';
3 import 'page2.dart';
4
5 class Page1 extends StatelessWidget {
6   const Page1({super.key});
7 }
```

```

8  @override
9  Widget build(BuildContext context) {
10    return Scaffold(
11      appBar: AppBar(
12        title: const Text('Page 1'),
13        centerTitle: true,
14      ),
15      body: Center(
16        child: ElevatedButton(
17          onPressed: () {
18            Navigator.of(context).push<void>(
19              MaterialPageRoute<void>(
20                builder: (context) => const Page2(),
21              ),
22            );
23          },
24          child: const Icon(Icons.navigate_next, size: 50),
25        ),
26      ),
27    );
28  }
29 }

```

`Navigator.of(context)` finds the nearest enclosing navigator. The `builder` receives a context for the new route and returns its screen. The `void` type says that this route is not expected to return a useful result.

2.3 Page 2: pop the route

```

1  // page2.dart
2  import 'package:flutter/material.dart';
3
4  class Page2 extends StatelessWidget {
5    const Page2({super.key});
6
7    @override
8    Widget build(BuildContext context) {
9      return Scaffold(
10        appBar: AppBar(
11          title: const Text('Page 2'),
12          centerTitle: true,
13        ),
14        body: Center(
15          child: ElevatedButton(
16            onPressed: () => Navigator.of(context).pop(),
17            child: const Icon(Icons.navigate_before, size: 50),
18          ),
19        ),
20      );
21    }
22  }

```

The second screen normally also receives an automatic AppBar back button. Both that button and our custom button return to Page 1. Do not use `pop` to leave the only route in this simple application.

Try it. Start on Page 1, tap the forward icon, read the Page 2 title, then tap the backward icon. Repeat several times. The route stack alternates between one route and two routes; it should not grow on every return.

3 BuildContext and icons

A `BuildContext` identifies a widget's location in the widget tree. It is not a container holding copies of all the application's settings. For example, `Theme.of(context)` looks up the enclosing theme. Display information comes from `MediaQuery.of(context)`. A navigator is found with `Navigator.of(context)`.

The context of `MyApp.build` is above the `MaterialApp` it returns. It cannot find the navigator created below it. The context of `Page1.build` is below that navigator and is suitable for our navigation callback. When using a context after an `await`, check that its widget is still mounted before using it.

`Icons.navigate_next` is an `IconData`; `Icon` is the widget that draws it. Its `size` is measured in logical pixels. An icon-only button should also have an accessible description, for example by wrapping it in a `Tooltip`. A labeled alternative is:

```
1 ElevatedButton.icon(  
2   onPressed: openPage2,  
3   icon:      const Icon(Icons.navigate_next),  
4   label:     const Text('Show salary'),  
5 )
```

A button with `onPressed: () {}` is enabled but does nothing. To disable it, use `onPressed: null`.

4 Second application: pass employee data

The input screen receives hours worked and an hourly rate. It creates an `Employee`, then opens a result screen displaying its net salary. We use a fixed 10% deduction as an **exercise assumption**, not as a real payroll rule.

File	Responsibility
<code>main.dart</code>	Starts the app; use the same entry point as the first example.
<code>employee.dart</code>	Stores valid hours and rate; computes the salary.
<code>page1.dart</code>	Owns controllers, checks input and pushes the result route.
<code>page2.dart</code>	Retrieves the employee and displays the result.

4.1 The employee model

```
1 // employee.dart  
2 class Employee {  
3   final int hours;  
4   final double rate;  
5   static const double tax = 0.10;  
6  
7   Employee(this.hours, this.rate) {  
8     if (hours <= 0 || !rate.isFinite || rate <= 0) {  
9       throw ArgumentError('Hours and rate must be positive.');10    }  
11  }  
12  
13  double netSalary() => hours * rate * (1 - tax);  
14  
15  @override  
16  String toString() =>
```

```

17     'Hours: $hours\n'
18     'Rate: ${rate.toStringAsFixed(2)}\n'
19     'Tax: ${((tax * 100).toStringAsFixed(0))}%\n'
20     'Net salary: ${netSalary().toStringAsFixed(2)}';
21 }

```

The fields are final: the result screen receives an object whose values cannot be reassigned. We reject zero as well as negative values, and reject non-finite rates such as infinity or NaN. Formatting to two decimal places makes the display readable; it does not turn binary floating-point arithmetic into exact accounting arithmetic.

4.2 Controllers and their life cycle

Page 1 now needs a `StatefulWidget`: its `State` owns two controllers for as long as the screen exists. Create the controllers once as state fields, use their `text` values when the user taps the button, and dispose of both when the state is permanently removed. `dispose` is a Flutter life-cycle callback, not a Dart destructor.

4.3 The complete input screen

```

1 // page1.dart -- replace the first example's file
2 import 'package:flutter/material.dart';
3 import 'employee.dart';
4 import 'page2.dart';
5
6 class Page1 extends StatefulWidget {
7   const Page1({super.key});
8
9   @override
10  State<Page1> createState() => _Page1State();
11 }
12
13 class _Page1State extends State<Page1> {
14   final _controllerHours = TextEditingController();
15   final _controllerRate = TextEditingController();
16
17   @override
18   void dispose() {
19     _controllerHours.dispose();
20     _controllerRate.dispose();
21     super.dispose();
22   }
23
24   void openPage2() {
25     final hours = int.tryParse(_controllerHours.text.trim());
26     final rate = double.tryParse(_controllerRate.text.trim());
27     if (hours == null || hours <= 0 || rate == null ||
28         !rate.isFinite || rate <= 0) {
29       ScaffoldMessenger.of(context).showSnackBar(
30         const SnackBar(
31           content: Text(
32             'Enter positive whole hours and a positive rate.',
33           ),
34         ),
35       );
36       return;
37     }
38
39     final employee = Employee(hours, rate);
40     Navigator.of(context).push<void>(
41       MaterialPageRoute<void>(

```

```

42     builder: (context) => const Page2(),
43     settings: RouteSettings(arguments: employee),
44   ),
45 );
46 }
47
48 @override
49 Widget build(BuildContext context) {
50   return Scaffold(
51     appBar: AppBar(title: const Text('Employee input')),
52     body: SingleChildScrollView(
53       padding: const EdgeInsets.all(24),
54       child: Column(
55         crossAxisAlignment: CrossAxisAlignment.stretch,
56         children: [
57           TextField(
58             controller: _controllerHours,
59             keyboardType: TextInputType.number,
60             decoration: const InputDecoration(
61               border: OutlineInputBorder(),
62               labelText: 'Hours worked',
63             ),
64           ),
65           const SizedBox(height: 16),
66           TextField(
67             controller: _controllerRate,
68             keyboardType: const TextInputType.numberWithOptions(
69               decimal: true,
70             ),
71             decoration: const InputDecoration(
72               border: OutlineInputBorder(),
73               labelText: 'Hourly rate (use a decimal point)',
74             ),
75           ),
76           const SizedBox(height: 16),
77           ElevatedButton.icon(
78             onPressed: openPage2,
79             icon: const Icon(Icons.navigate_next),
80             label: const Text('Show salary'),
81           ),
82         ],
83       ),
84     ),
85   );
86 }
87 }

```

`tryParse` returns null for invalid numeric text instead of throwing a parsing exception. A numeric keyboard is only a hint: it does not validate pasted or typed input. This example accepts rates written with a decimal point. The scroll view helps keep the fields reachable when the keyboard reduces the available height.

4.4 RouteSettings and arguments

The `settings` belong to the route; `arguments` carries our employee object. This passes an object reference inside the application, not serialized network data. The route builder returns `Page2`, while the settings separately hold the employee:

```
1 settings: RouteSettings(arguments: employee),
```

Because `arguments` has type `Object?`, the receiving screen must agree with the sender about the object's type. A direct cast works only when that agreement is respected. We check the type

so that an incorrectly opened route has a useful fallback.

4.5 The complete result screen

```

1 // page2.dart -- replace the first example's file
2 import 'package:flutter/material.dart';
3 import 'employee.dart';
4
5 class Page2 extends StatelessWidget {
6   const Page2({super.key});
7
8   @override
9   Widget build(BuildContext context) {
10    final arguments = ModalRoute.of(context)?.settings.arguments;
11    return Scaffold(
12      appBar: AppBar(title: const Text('Employee salary')),
13      body: SingleChildScrollView(
14        padding: const EdgeInsets.all(24),
15        child: Column(
16          crossAxisAlignment: CrossAxisAlignment.stretch,
17          children: [
18            Text(
19              arguments is Employee
20                ? arguments.toString()
21                : 'No employee data was provided.',
22              style: const TextStyle(
23                fontSize: 24,
24                fontWeight: FontWeight.bold,
25              ),
26            ),
27            const SizedBox(height: 16),
28            ElevatedButton.icon(
29              onPressed: () => Navigator.of(context).pop(),
30              icon: const Icon(Icons.navigate_before),
31              label: const Text('Back to input'),
32            ),
33          ],
34        ),
35      ),
36    );
37  }
38 }

```

`ModalRoute.of(context)` finds the enclosing route. The null-aware `?.` avoids assuming that a route is always present. The `is Employee` check promotes the value to `Employee` before calling its display method.

Expected result. Enter 40 hours and a rate of 25. Gross salary is $40 \times 25 = 1000$; the exercise's deduction is 100; net salary is 900.00. Return to the input screen: both fields keep their values. Empty text, abc, zero, negative input and a non-finite rate should show a message and leave you on the input screen.

5 Going further: constructor parameters

For a destination whose required data is already known, a typed constructor makes the dependency explicit. This is an alternative to the route-settings version, not an additional step required by it.

```

1 class SalaryPage extends StatelessWidget {
2   const SalaryPage({super.key, required this.employee});

```

```

3   final Employee employee;
4
5   @override
6   Widget build(BuildContext context) => Scaffold(
7     appBar: AppBar(title: const Text('Salary')),
8     body: Center(child: Text(employee.toString())),
9   );
10 }
11
12 // In the input screen, after validating the employee:
13 Navigator.of(context).push<void>(
14   MaterialPageRoute<void>(
15     builder: (context) => SalaryPage(employee: employee),
16   ),
17 );

```

Here the analyzer checks the parameter type. There is no runtime cast and no need to read `ModalRoute`. The destination is not created with `const` because the employee is a runtime value.

6 Named routes: from a category to an item

The runnable example `code/week06/examples/lib/07_named_routes.dart` adds a three-screen flow: categories, the items in one category, then the details of one item. It uses local data and needs no network images. Run it from `code/week06/examples` with `flutter run -t lib/07_named_routes.dart`.

6.1 A route name is a key

`pushNamed` requests a route by a string such as `/category`. A static `const` `routeName` on a screen avoids repeating that string. `MaterialApp.routes` maps names to widget-builder functions; it is a **Map**, not a list of routes. The example registers `/` as its initial screen, then generates routes that require arguments.

```

1  initialRoute: '/',
2  routes: {'/': (context) => const CategoriesPage()},
3
4  // When a category is tapped:
5  Navigator.of(context).pushNamed<void>(
6    CategoryPage.routeName,
7    arguments: category,
8  );

```

Named routes are useful for reading existing Flutter apps and for this small exercise. Flutter does not recommend them for most new applications; use direct routes for simple navigation, or a Router-based solution when URL navigation is required.

6.2 Generate a route and validate its arguments

When a name is absent from the route table, `onGenerateRoute` can build a route from `RouteSettings`. In our example it checks the argument, then passes it to a typed screen constructor. Keep `settings: settings` on the returned route to preserve its name and arguments.

```

1  final argument = settings.arguments;
2  if (settings.name == CategoryPage.routeName &&
3      argument is String &&
4      ['Food', 'Drinks'].contains(argument)) {
5    return MaterialPageRoute<void>(
6      settings: settings,

```

```

7     builder: (context) => CategoryPage(category: argument),
8   );
9 }

```

The full example gives invalid arguments a visible error screen. For other names its generator returns `null`, allowing `onUnknownRoute` to build an error route. That fallback is *another pushed route*; it does not mean the navigator has stayed on the home screen. It does not handle a failed initial-route request in the same way as a later named push.

Initial routes. `home` defines the `/` route; do not also define `/` in `routes`. A non-root `initialRoute` can build an initial stack with `/` beneath the requested route. This explains a startup screen that can be dismissed to reveal `home`. It is not an authentication boundary: a login-looking screen and `fullscreenDialog: true` do not enforce access control.

6.3 Filter a list, then find one item

The pasted news examples suggest a reusable pattern: select a category, filter a collection, then open a detail by ID. Here we use menu entries to connect that pattern to the restaurant app.

```

1 final entries = menuEntries
2   .where((entry) => entry.category == widget.category)
3   .toList();
4
5 // In the route generator, after checking the ID exists:
6 final entry = menuEntries.firstWhere(
7   (entry) => entry.id == argument,
8 );

```

`where` produces an iterable of all matching items; `toList` makes it convenient to index in `ListView.builder`. `firstWhere` returns one item and throws when none matches unless a fallback is supplied. Our generator checks existence before calling it. An ID needs a backing collection; passing a whole object through a typed constructor avoids that lookup.

An `enum` represents a fixed set of typed values, not an arbitrary integer or string. Our `ItemKind` has `meal` and `drink`; an exhaustive switch gives each a readable label.

```

1 final kindText = switch (entry.kind) {
2   ItemKind.meal => 'Meal',
3   ItemKind.drink => 'Drink',
4 };

```

6.4 Return a result to the previous screen

Passing an argument sends data forward. Returning a result sends data back: the future from `pushNamed<String>` completes when the detail route is popped. The detail's choice button uses `pop<String>(entry.title)`. The ordinary back button returns `null`.

```

1 Future<void> _openDetail(MenuEntry entry) async {
2   final result = await Navigator.of(context).pushNamed<String>(
3     DetailPage.routeName,
4     arguments: entry.id,
5   );
6   if (!mounted || result == null) return;
7   setState(() => _chosen = result);
8 }

```

The `mounted` check matters because the calling screen may have been removed while awaiting the result. Cancellation preserves the previous selection. Try Food, Burger, **Choose this item**: the list shows **Chosen: Burger**. Open Salad and press Back: the previous choice remains.

6.5 Connect the restaurant app to the lecture

In `code/week06/flutter_examples`, the cart button now pushes `/selected` with a `List<Item>`. The route builder reads `ModalRoute.of(context)?.settings.arguments`, checks its type, and passes the list to `SelectedItemsScreen`. Its AppBar back button pops the route; the menu's selections and total remain in the underlying `Home` state. This is navigation between two screens, rather than replacing the body of one screen.

6.6 Corrections to the supplementary reading

- **Route versus widget:** Route is a navigation object, not a subclass of `Widget`. Its builder creates screen widgets. `Navigator` is a widget, and `Navigator.of(context)` returns its state.
- **Context versus payload:** arguments are stored in route settings. Context locates the route; it does not contain the payload itself.
- **Map versus List:** `{'id': id, 'title': title}` is a Map. A `List<Item>` is a different payload. Both are objects, and neither is required for every navigation.
- **Null safety:** `!` asserts a non-null value at runtime; it is not a mandatory null check. Prefer a checked type or a typed constructor when appropriate.
- **Modern text themes:** use `bodyMedium`, `bodyLarge` and `titleLarge` instead of the old `bodyText2`, `bodyText1` and `headline6` names in the excerpts.

7 Corrections and common errors

- **Returning by pushing Page 1:** this adds another route. Use `pop` to reveal the existing input screen.
- **Wrong context:** a context above `MaterialApp` cannot find the navigator it creates. Navigate from a screen below it.
- **Stateful versus stateless:** select a stateful widget when the screen needs owned mutable state or resources. Being recreated on navigation is not the definition of a stateless widget.
- **Missing arguments or an incorrect cast:** An unchecked `as Employee` cast assumes the right object was supplied; using `!` on the route lookup also assumes a route exists. Check them or pass a typed constructor parameter.
- **Controllers created in build:** rebuilds can replace them and lose input. Keep them in state fields and dispose of them.
- **Errors printed only in the console:** users need visible feedback. Show a message, or reuse week 5's form validators.
- **Zero described as negative:** the model rejects all non-positive values, including zero. Match the error message to the condition.

7.1 Navigation on the web

`Navigator` works on Flutter web, but simple imperative pushes do not by themselves define a complete URL and deep-linking strategy. For applications that require browser history, forward navigation, deep links or several navigators, Flutter's navigation guidance recommends a Router-based approach, often through a routing package such as `go_router`. Our two-screen examples teach the basic route stack; they do not configure URL-based navigation.

8 Exercises

1. Add Page 3 to the first application. Push it from Page 2, then pop twice. Draw the stack after each operation.
2. Replace the forward icon-only button with `ElevatedButton.icon` and a text label.
3. Push another Page 1 instead of popping Page 2. Observe how many back operations are needed, then restore the correct behavior.
4. In the salary application, test 40 hours at 25 and 10 hours at 12.5. Expected net salaries are 900.00 and 112.50.
5. Test blank fields, letters, zero, negative numbers, fractional hours and the rate `NaN`. Explain why each is rejected.
6. Convert the input screen to a `Form` with `TextFormField`s, using the validation techniques from week 5.
7. Replace `RouteSettings` with the typed `SalaryPage` constructor. Identify which errors the analyzer can now catch.
8. In example 07, add a third category and one item. Test the filtered list, detail screen and returned choice.
9. Call example 07 with an unknown route name, a missing item ID and a wrong argument type. Confirm each shows an error screen that you can leave with `Back`.
10. Add a clear button on the input screen. Verify that returning from the salary screen preserves input until that button is tapped.

9 Summary

Idea	What to remember
Route stack	The newest route is on top; popping reveals the previous route.
<code>push</code> / <code>pop</code>	Open a new screen / return from the current one.
<code>BuildContext</code>	A location used to look up an enclosing navigator or route.
<code>RouteSettings</code>	Carries route metadata, including an optional arguments object.
<code>ModalRoute.of</code>	Reads the enclosing route and its settings.
Controllers	Owned by state; created once and disposed when state is removed.
Validation	Check numeric input before constructing data or navigating.
Typed constructor	Makes the destination's required data explicit to the analyzer.

Further reading

- [Flutter cookbook: Navigate to a new screen and back.](#)
- [Flutter cookbook: Send data to a new screen.](#)
- [Flutter: Navigation and routing.](#)

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-10-05